POC05 · Saga Pattern + Circuit Breaker

Guía de implementación y verificación · Go / PostgreSQL / Postman / DataGrip

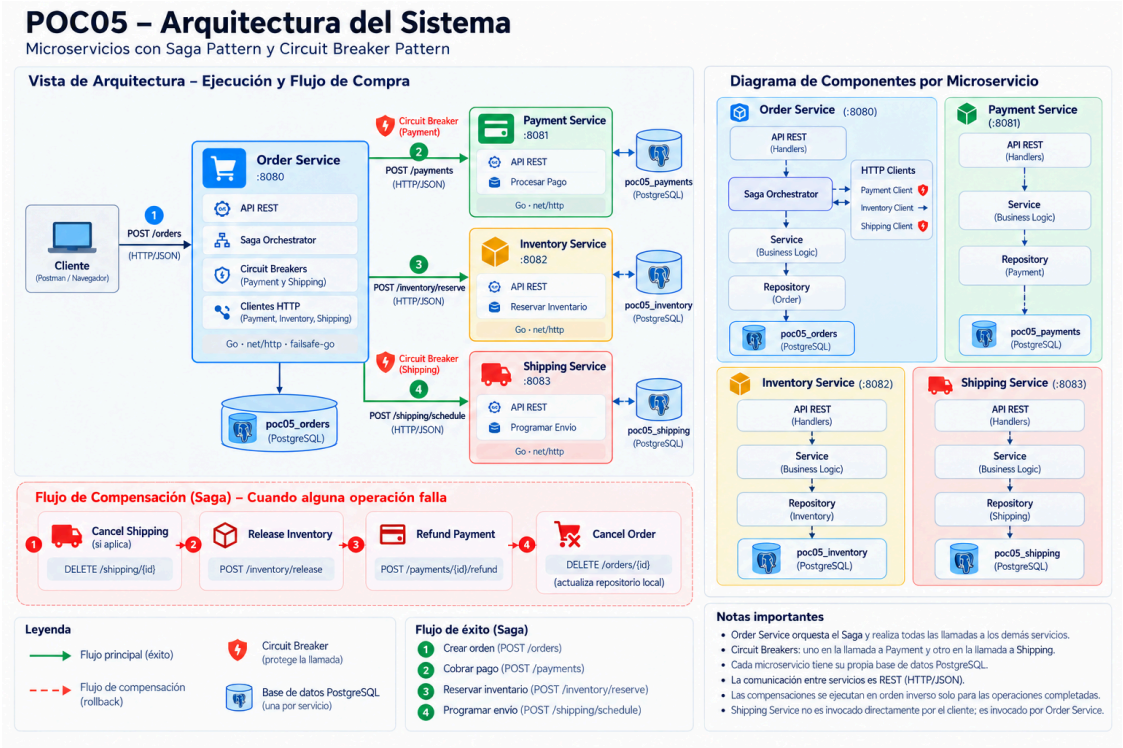
SECCIÓN 1: Marco conceptual e implementación del POC

1.1 Qué resuelve esta POC

Presentación

Esta prueba de concepto modela un flujo de compra distribuida con **cuatro microservicios en Go**: **Order**, **Payment**, **Inventory** y **Shipping**. Cada servicio conserva su propia base de datos PostgreSQL y se comunica por **HTTP/JSON**. El objetivo es demostrar dos patrones complementarios:

- **Saga**, para coordinar una compra de punta a punta y compensar pasos anteriores si ocurre un fallo.
- **Circuit Breaker**, para proteger a Order frente a errores repetidos al invocar **Payment** y **Shipping**.



Arquitectura de POC05: Order orquesta Saga y protege llamadas a Payment y Shipping con Circuit Breaker.

PUNTO CLAVE

- **Order** expone POST /orders y concentra la orquestación.
- **Payment** cobra y reembolsa.
- **Inventory** reserva y libera stock.
- **Shipping** programa envíos.
- Las bases independientes son: poc05_orders, poc05_payments, poc05_inventory y poc05_shipping.

REFERENCIA EN CÓDIGO

- Bootstrap general: **services/order/cmd/main.go**, main(), líneas **15-46**.
- Creación de clientes y Circuit Breakers: **services/order/cmd/main.go**, líneas **34-43**.
- Entrada HTTP de Order: **services/order/internal/order/handler.go**, create(), líneas **40-54**.

1.2 Saga Pattern aplicado a la compra distribuida

Idea del patrón

Saga coordina una secuencia de **transacciones locales**. Cada servicio escribe solo en su propia base. Si todos los pasos salen bien, la compra termina completada. Si un paso falla, se ejecutan **compensaciones** en orden inverso sobre lo que ya se realizó.



Saga Pattern: happy path y compensation route (rollback) aplicada.

PSEUDOCÓDIGO DEL FLUJO

```
create order (PENDING)
charge payment
reserve inventory
schedule shipping
complete order

if step fails:
    if inventory reserved -> release inventory
    if payment charged   -> refund payment
    cancel order
```

REFERENCIA EN CÓDIGO

- La compra se coordina en **services/order/internal/order/service.go**.
- El flujo principal está en Create(), líneas **54-103**.
- La compensación está en failSaga(), líneas **121-183**.
- La persistencia de Order está en **services/order/internal/order/repository.go**.
- En Inventory, las operaciones clave están en Reserve() líneas **29-84** y Release() líneas **87-145**, archivo **services/inventory/internal/inventory/repository.go**.
- Endpoints remotos invocados por Order: **services/order/internal/order/client.go**, métodos Charge(), Reserve(), Schedule(), Refund(), Release().

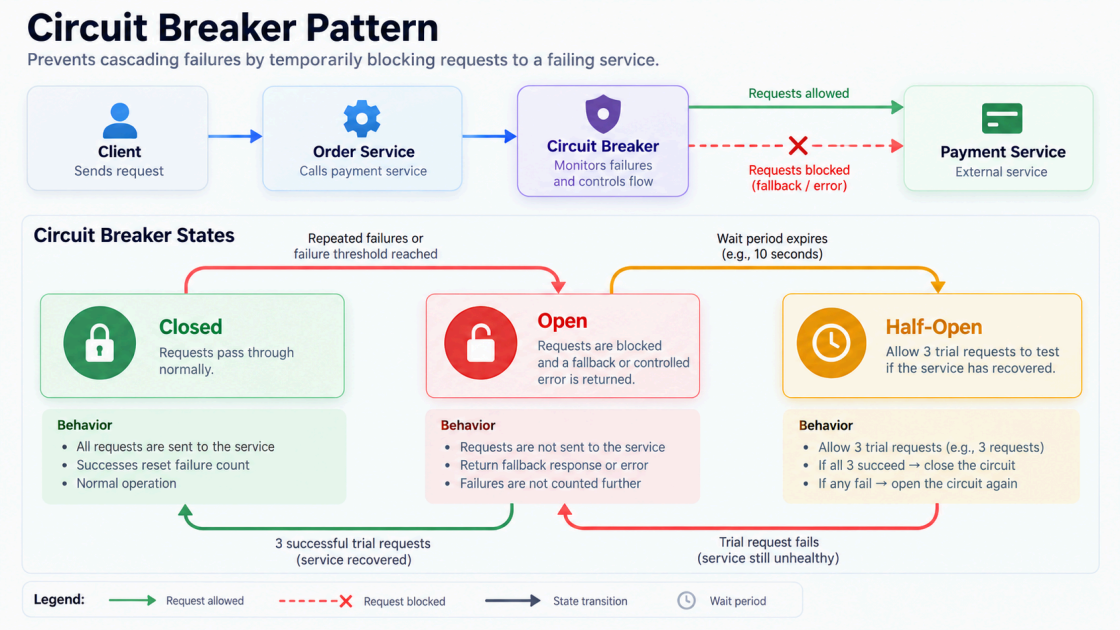
EXPLICACIÓN OPERATIVA

- En el **happy path**, vemos: PENDING → CHARGED → RESERVED → SCHEDULED → COMPLETED.
- Si **Inventory** falla, Order compensa con **Refund Payment** y deja la orden **CANCELLED**.
- Si **Shipping** falla después de cobrar y reservar, Order compensa con **Release Inventory** → **Refund Payment** → **Cancel Order**.
- **Cancel Shipping** es solo un paso opcional si el envío ya existía. En esta POC, el escenario principal de fallo de Shipping ocurre **antes** de que el envío sea creado.

1.3 Circuit Breaker aplicado a Payment y Shipping

Idea del patrón

Circuit Breaker evita insistir sobre un servicio remoto que está fallando. En esta POC vive **dentro de Order** y protege dos llamadas: la de **Payment** y la de **Shipping**. No protege Inventory y tampoco participa en las compensaciones.



*Circuit Breaker: estados **Closed**, **Open** y **Half-Open** con ventana de **10 segundos** y **3 éxitos de recuperación**.*

Estados que debemos entender

- **Closed**: las solicitudes pasan normalmente y el breaker registra resultados.
- **Open**: las nuevas solicitudes protegidas se bloquean sin llamar al servicio remoto.
- **Half-Open**: después del tiempo de espera, se permiten algunas solicitudes de prueba para validar la recuperación.

ALGORITMO RESUMIDO

```
1. Iniciar en Closed.
2. Dejar pasar solicitudes y contar ejecuciones/fallos técnicos.
3. Si se alcanza el umbral configurado, pasar a Open.
4. Mientras está Open, bloquear llamadas protegidas.
5. Tras pasar el tiempo de espera configurado (10 seg.), permitir pruebas en Half-Open.
6. Si la cant. De pruebas configurada (3 pruebas) salen bien, volver a Closed.
7. Si una prueba falla, volver a Open (reset del tiempo de espera configurado) y se repite.
```

Configuración implementada

```
WithFailureThresholdRatio(5, 10)

WithDelay(10 * time.Second)

WithSuccessThreshold(3)

HandleIf(isTechnicalFailure)
```

REFERENCIA EN CÓDIGO

- Implementación del breaker: **services/order/internal/order/circuit_breaker.go**.
- Creación y configuración: newBreaker(), líneas **54-69**.
- Clasificación de error técnico: isTechnicalFailure(), líneas **71-81**.
- Diagnóstico del estado: snapshot(), líneas **91-102**.
- Uso al cobrar: **services/order/internal/order/client.go**, Charge(), líneas **70-84**.
- Uso al programar envío: **services/order/internal/order/client.go**, Schedule(), líneas **107-120**.
- Refund y Release van sin breaker: Refund() líneas **86-89** y Release() líneas **100-105**.
- Endpoint de diagnóstico: **services/order/internal/order/handler.go**, líneas **25-39**.

Qué significa esto en el POC

- El circuito **abre** cuando alcanza **5 fallos en una ventana de 10 ejecuciones**.
- Permanece **Open durante 10 segundos**.
- Para recuperarse necesita **3 éxitos consecutivos** en **Half-Open**.
- Solo cuenta como fallo técnico lo que define isTechnicalFailure(): típicamente **errores 5xx** o de red, no errores funcionales como un 409 por stock insuficiente.

1.4 Relación entre ambos patrones en esta implementación

Cierre de la sección conceptual

Saga y Circuit Breaker resuelven problemas distintos y por eso se complementan. **Saga** mantiene consistencia entre pasos ya ejecutados. **Circuit Breaker** mejora resiliencia evitando llamadas repetidas a servicios remotos inestables.

En términos prácticos:

- Si falla **Payment**, Circuit Breaker puede bloquear el cobro antes de que la Saga avance.
- Si falla **Shipping**, la Saga todavía puede compensar lo ya realizado: liberar inventario y reembolsar el pago.
- Por eso, en la demostración veremos casos donde solo actúa Saga y otros donde actúan **Saga + Circuit Breaker** al mismo tiempo.

SECCIÓN 2: Demostración funcional guiada

> **Apertura** «**Ahora veremos los dos patrones funcionando en una compra. En Postman enviaremos las solicitudes, en DataGrip comprobaremos sus efectos y en la terminal de Order observaremos los cambios del Circuit Breaker.**»

2.1 Preparación única: datos limpios y servicios

> **NOTA CRÍTICA** El **reset** elimina los datos de prueba de las cuatro bases poc05_*. Solo ejecutarlo con las cuatro APIs detenidas; no elimina tablas. No **reiniciar** bases entre los flujos.

PASO 1 · Detener las 4 terminales con **Ctrl+C**. En la raíz del proyecto ejecutar bash POC05_Reset_Demo.sh y confirmar **REINICIAR**. Resultado: **SKU-001 = 100**.

PASO 2 · Levantar las cuatro APIs (una terminal por servicio, situada en la carpeta indicada):

> **T1 · Payment [services/payment]** PORT=8081
> DATABASE_URL='postgres://postgres@127.0.0.1:5432/poc05_payments?sslmode=disable' DEMO_MODE=true go run ./cmd
>
> **T2 · Inventory [services/inventory]** PORT=8082
> DATABASE_URL='postgres://postgres@127.0.0.1:5432/poc05_inventory?sslmode=disable' go run ./cmd
>
> **T3 · Shipping [services/shipping]** PORT=8083
> DATABASE_URL='postgres://postgres@127.0.0.1:5432/poc05_shipping?sslmode=disable' DEMO_MODE=true go run ./cmd
>
> **T4 · Order [services/order]** PORT=8080 DATABASE_URL='postgres://postgres@127.0.0.1:5432/poc05_orders?sslmode=disable' PAYMENT_URL='http://127.0.0.1:8081' INVENTORY_URL='http://127.0.0.1:8082' SHIPPING_URL='http://127.0.0.1:8083' go run ./cmd

Abrir cada terminal en la carpeta services/<servicio> indicada (o entrar allí una sola vez con cd). Los comandos sí son ejecutables tal cual en esa carpeta; ~/.pgpass aporta la contraseña.

PASO 3 · **Postman**, carpeta 00: **EJECUTAR** **00.01–00.04** (health 200), **00.05–00.06** (**enabled:false**), **00.07** (ambos **closed**, **executions:0**), **00.08** (**stock 100**).

> **VERIFICACIÓN** Al iniciar, orders, payments, reservations y shipments están vacías; items contiene SKU-001 con 100.

2.2 DataGrip: consultas A-D

Abrir por doble clic en la tabla y refrescar; o ejecutar estas consultas guardadas en cada conexión. "A+B+C+D" significa revisar las cinco tablas en las cuatro bases.

> **A · Orders [poc05_orders]** SELECT id, status, payment_id, shipping_id, failure_reason FROM orders **ORDER** BY id DESC LIMIT 20;
>
> **B · Payments [poc05_payments]** SELECT id, order_id, amount_cents, status FROM payments **ORDER** BY id DESC LIMIT 20;
>
> **C · Inventory [poc05_inventory]** SELECT sku, available_quantity FROM items; SELECT order_id, quantity, status FROM reservations **ORDER** BY order_id DESC LIMIT 20;
>
> **D · Shipping [poc05_shipping]** SELECT id, order_id, status FROM shipments **ORDER** BY id DESC LIMIT 20;

2.3 Saga: cuatro escenarios

> **Apertura** «**Primero demostraremos Saga. Order coordina el cobro, la reserva y el envío. Si algún paso falla, compensa las operaciones anteriores para no dejar una compra a medias.**»

S1 Compra exitosa: Happy path

> **Contexto** «**Un cliente compra dos unidades. Vamos a comprobar que el pago, la reserva y el envío se registran correctamente y que la orden termina completada.**»

PASO 1 · Estado inicial en **DataGrip**: ver A+B+C+D: sin operaciones; **stock 100**.

PASO 2 · Postman **S1.01**: enviar UNA vez, cantidad 2 → **HTTP 201**.

PASO 3 · Revisar resultado en **DataGrip**: refrescar A+B+C+D.

VERIFICACIÓN 1 orden **COMPLETED** · 1 pago **CHARGED** · 1 reserva **RESERVED** (2 unidades) · 1 envío **SCHEDULED** · **stock 100 → 98**

Interpretación «**La orden quedó completada, el pago se cobró, el inventario se reservó y el envío se programó. Como todo salió bien, no hubo nada que revertir.**»

S2 Inventario insuficiente

Contexto «**¿Qué ocurre si un cliente pide 999 unidades y solo tenemos 98? El cobro se procesa primero, pero el inventario rechaza la reserva. Comprobaremos cómo Saga revierte el cobro.**»

PASO 1 · Estado inicial en **DataGrip**: ver A+B+C+D; **stock 98**.

PASO 2 · Postman **S2.01**: enviar UNA vez, cantidad 999 → **HTTP 409**, insufficient_stock.

PASO 3 · Revisar resultado en **DataGrip**: refrescar A+B+C+D.

VERIFICACIÓN Nueva orden **CANCELLED** y pago **REFUNDED**; sin reserva ni envío nuevos; stock sigue 98

Interpretación «**La orden quedó cancelada y el pago reembolsado. No se creó una reserva ni disminuyó el stock: Saga evitó que una compra sin inventario dejara un cobro pendiente.**»

S3 Fallo simulado de Payment

Contexto «**Supongamos que el servicio de pagos está temporalmente fuera de servicio. Lo simulamos con el control de fallos: la compra recibirá un 503 antes de registrar el cobro.**»

PASO 1 · Estado inicial en **DataGrip**: ver A+B+C+D.

PASO 2 · Postman **S3.01**: **Payment** fallo ON → **enabled:true**; esperar respuesta.

PASO 3 · **S3.02**: enviar compra UNA vez → **HTTP 503**, payment_unavailable.

PASO 4 · **S3.03**: **Payment** fallo OFF → **enabled:false**.

PASO 5 · Revisar resultado en **DataGrip**: refrescar A+B+C+D.

VERIFICACIÓN Nueva orden **CANCELLED**; ningún pago, reserva ni envío nuevos; **stock 98**.

Interpretación «**Como el cobro no pudo realizarse, la orden se canceló. No encontramos pagos, reservas ni envíos nuevos porque la Saga se detuvo en el primer paso.**»

S4 Fallo simulado de Shipping

Contexto «**Ahora imaginemos que no se puede programar el envío. El pago y la reserva ya ocurrieron, así que Saga tendrá que deshacer ambos pasos. Activaremos esa falla para verlo.**»

PASO 1 · Estado inicial en **DataGrip**: ver A+B+C+D.

PASO 2 · Postman **S4.01**: **Shipping** fallo **ON** → **enabled:true**; esperar.

PASO 3 · **S4.02**: enviar compra **UNA vez** → **HTTP 503**, shipping_unavailable.

PASO 4 · **S4.03**: **Shipping** fallo OFF → **enabled:false**.

PASO 5 · Revisar resultado en **DataGrip**: refrescar A+B+C+D; mostrar también logs de **Terminal 4**.

VERIFICACIÓN Orden **CANCELLED** · pago **REFUNDED** · reserva **RELEASED** · sin envío nuevo · stock sigue 98

VERIFICACIÓN · Terminal 4 (Order): step=compensate_inventory status=**RELEASED**; step=compensate_payment status=**REFUNDED**

Interpretación «**El envío no se programó. La reserva aparece liberada y el pago reembolsado; la orden terminó cancelada. Es la compensación en orden inverso que acabamos de provocar.**»

2.4 Circuit Breaker: reglas visibles durante la prueba

> Apertura «**Circuit Breaker funciona como un interruptor de protección. En Closed permite las llamadas y mide los fallos; en Open deja de llamar al servicio que falla; y en Half-Open prueba si ya se recuperó. En nuestra configuración abre con cinco fallos de diez, espera diez segundos y necesita tres éxitos para cerrar.**»

Hay dos circuitos separados, **Payment** y **Shipping**, ambos en la memoria de **Order**. La falla demo hace responder 503; el breaker observa ese error técnico y decide abrir. No guarda sus estados en PostgreSQL: se prueban con Pxx/Hxx, GET /demo/circuit-breakers y logs en **Terminal 4**.

> **NOTA CRÍTICA** ANTES de **Payment** y ANTES de **Shipping**: **reiniciar Terminal 4** · **Order**, con **Ctrl+C** y el mismo comando. Verificar luego **P01**/H01: **executions:0**. **Reiniciar Payment** o **Shipping** NO pone a cero los circuitos.

2.5 Circuit Breaker de Payment - Postman 02

> Transición «**Empezaremos por Payment. Primero veremos compras normales, después provocaremos fallos hasta que el circuito bloquee el cobro y, por último, comprobaremos su recuperación.**»

P · Fase 1 Closed: compras normales

> Contexto «**Con Payment funcionando, realizaremos cinco compras. El circuito debe registrar cinco éxitos y permanecer Closed, es decir, permitiendo los cobros.**»

PASO 1 · Preparación: ambos DEMO OFF; **reiniciar** SOLO **Terminal 4** · **Order** (**Ctrl+C** y volver a ejecutar go run ./cmd desde services/order).

PASO 2 · **P01**: enviar **una vez** → **payment**.state=**closed** y **executions=0**. Si no da cero, detenerse.

PASO 3 · ANTES - **DataGrip**: ver A+B+C+D; **stock 98**.

PASO 4 · **P02**: enviar **EXACTAMENTE 5 veces**, de forma secuencial; esperar cada **HTTP 201**. Contar 1/5, 2/5, 3/5, 4/5, 5/5.

PASO 5 · Revisar resultado en **DataGrip**: refrescar A+B+C+D.

PASO 6 · **P03**: enviar una vez y revisar diagnóstico.

> **VERIFICACIÓN** 5 órdenes **COMPLETED** + 5 pagos **CHARGED** + 5 reservas **RESERVED** + 5 envíos **SCHEDULED**; **stock 98 → 93**. **P03**: **payment closed**, **executions=5**, **failures=0**

> Interpretación «**Vemos cinco órdenes completas y cinco pagos cobrados. El circuito registra cinco ejecuciones, cero fallos y sigue Closed: no hay motivo para bloquear Payment.**»

P · Fase 2 Open: detectar fallos y rechazar

> Contexto «**Ahora supongamos que el proveedor de pagos empieza a fallar. Activaremos respuestas 503 y enviaremos cinco compras fallidas para que el circuito detecte el problema y se abra.**»

> **NOTA CRÍTICA** SECUENCIA CRÍTICA: tras la quinta **P05**, **P06** y **P07** sin pausas ni **DataGrip**. El intervalo **Open** es de **10 s**. Explicar y mostrar tablas DESPUÉS.

PASO 1 · Estado inicial en **DataGrip**: ver A+B (opcional C+D); **stock 93**.

PASO 2 · **P04**: **Payment** fallo ON → **enabled:true**.

PASO 3 · **P05**: enviar **EXACTAMENTE 5 veces**, una por una; cada respuesta **HTTP 503**, payment_unavailable.

PASO 4 · **P06**: ENVIAR **INMEDIATAMENTE** al responder la quinta **P05** → **HTTP 503**, payment_circuit_open.

PASO 5 · **P07**: **ENSEGUIDA** consultar diagnóstico → **payment**.**open**; **executions=10**, **failures=5**, **failure_rate=0.5**.

PASO 6 · Revisar resultado en **DataGrip**: refrescar A+B. Mostrar logs de **Terminal 4**.

> **VERIFICACIÓN** 6 órdenes **CANCELLED** (cinco fallos y un rechazo); sin nuevos pagos ni reservas; **stock 93**. Log: **closed → open**. **P06** no aumenta las 10 ejecuciones del circuito.
>
> **VERIFICACIÓN · Terminal 4 (Order):** circuit_breaker=**payment** old_state=**closed** new_state=**open**

> **Interpretación** «**Las primeras cinco compras recibieron un error del servicio de pagos. Con esos cinco fallos, el circuito abrió; la sexta compra ya fue rechazada por Order, sin llamar a Payment. Saga canceló las órdenes y no llegó a reservar inventario.**»

P · Fase 3 Half-Open y recuperación

> **Contexto** «**Restableceremos Payment y esperaremos once segundos. Después, el circuito dejará pasar tres compras de prueba; si todas funcionan, volverá a Closed.**»

PASO 1 · **P08**: **Payment** fallo **OFF** → **enabled:false**.

PASO 2 · **Esperar 11 segundos** completos.

PASO 3 · **P09**: consultar diagnóstico; puede continuar **open** hasta la siguiente operación protegida.

PASO 4 · **P10** → **P11**: compra 1/3 (**HTTP 201**), luego diagnóstico **half-open**, 1 éxito.

PASO 5 · **P12** → **P13**: compra 2/3 (**HTTP 201**), luego diagnóstico **half-open**, 2 éxitos.

PASO 6 · **P14** → **P15**: compra 3/3 (**HTTP 201**), luego diagnóstico **closed**.

PASO 7 · Revisar resultado en **DataGrip**: refrescar A+B+C+D y mostrar logs de **Terminal 4**.

> **VERIFICACIÓN** 3 órdenes/pagos/reservas/envíos nuevos exitosos; **stock 93 → 90**. Logs: **open → half-open → closed**; al cerrar puede reiniciarse la ventana de métricas.
>
> **VERIFICACIÓN · Terminal 4 (Order):** circuit_breaker=**payment** old_state=**open** new_state=**half-open**; ... **half-open** new_state=**closed**

> **Interpretación** «**Esperar no cerró el circuito por sí solo. La primera compra lo llevó a Half-Open y, tras tres éxitos consecutivos, volvió a Closed. Así se comprueba que Payment ya responde.**»

2.6 Circuit Breaker de Shipping - Postman 03

> **Transición** «**Ahora probaremos Shipping. Aquí se verá la relación entre ambos patrones: si el circuito bloquea el envío, Saga todavía debe liberar el inventario y devolver el pago que ya se realizó.**»

> **NOTA CRÍTICA** OBLIGATORIO: **reiniciar Terminal 4 · Order** antes de **H01**. La prueba anterior también pasó por **Shipping** y dejó métricas acumuladas; **H01** debe mostrar **shipping.closed** y **executions=0**. No **reiniciar** PostgreSQL.

H · Fase 1 Closed: envíos normales

> **Contexto** «**Con Shipping funcionando normalmente, haremos cinco compras y comprobaremos que sus envíos se programan y que el circuito continúa Closed.**»

PASO 1 · Preparación: ambos DEMO OFF; **reiniciar** SOLO **Order** y mantener **Terminal 4** visible.

PASO 2 · **H01**: una vez → **shipping.closed** y **executions=0**. Si no muestra cero, NO avanzar.

PASO 3 · ANTES — **DataGrip**: ver A+B+C+D; **stock 90**.

PASO 4 · **H02**: **EXACTAMENTE 5 veces** secuenciales, cada una **HTTP 201**; contar cada envío.

PASO 5 · Revisar resultado en **DataGrip**: refrescar A+B+C+D.

PASO 6 · **H03**: una vez → **shipping.closed**, **executions=5**, **failures=0**.

> **VERIFICACIÓN** 5 órdenes, 5 pagos, 5 reservas y 5 envíos nuevos; **stock 90 → 85**

> **Interpretación** «**Las cinco compras completaron sus envíos. Shipping registra cinco éxitos y su circuito sigue Closed, porque no hay fallos que justifiquen bloquearlo.**»

H · Fase 2 Open: bloqueo de Shipping + Saga

> **Contexto** «**Simularemos que Shipping deja de responder. Tras cinco fallos, su circuito abrirá; aun así, como el pago y la reserva suceden antes del envío, comprobaremos que Saga los compensa.**»

> **NOTA CRÍTICA** SECUENCIA CRÍTICA: quinta **H05** → **H06** → **H07 inmediatamente**. No abrir **DataGrip** hasta registrar el diagnóstico.

PASO 1 · Estado inicial en **DataGrip**: ver A+B+C+D; **stock 85**.

PASO 2 · **H04**: **Shipping** fallo ON → **enabled:true**.

PASO 3 · **H05**: **EXACTAMENTE 5 veces** secuenciales, cada respuesta **HTTP 503**, shipping_unavailable.

PASO 4 · **H06**: **INMEDIATAMENTE** tras quinta **H05** → **HTTP 503**, shipping_circuit_open.

PASO 5 · **H07**: **ENSEGUIDA** diagnóstico **shipping.open**, **executions=10**, **failures=5**, **failure_rate=0.5**.

PASO 6 · Revisar resultado en **DataGrip**: refrescar A+B+C+D y ver logs de **Terminal 4**.

> **VERIFICACIÓN** 6 órdenes **CANCELLED**; 6 pagos **REFUNDED** y 6 reservas **RELEASED**; sin nuevos envíos; stock sigue 85. Logs: circuito **closed** → **open**, compensate_inventory y compensate_payment.
>
> **VERIFICACIÓN · Terminal 4 (Order):** circuit_breaker=**shipping** old_state=**closed** new_state=**open**; step=compensate_inventory status=**RELEASED**; step=compensate_payment status=**REFUNDED**

> **Interpretación** «**La sexta compra ya no llegó a Shipping: Circuit Breaker bloqueó esa llamada. Pero Payment e Inventory habían trabajado antes, así que Saga liberó las seis reservas y reembolsó los seis pagos. Así vemos la protección y la compensación trabajando juntas.**»

H · Fase 3 Half-Open y recuperación de Shipping

> **Contexto** «**Ahora Shipping vuelve a funcionar. Tras la espera, realizaremos tres compras de prueba para que el circuito verifique los envíos y regrese a Closed.**»

PASO 1 · **H08**: **Shipping** fallo OFF → **enabled:false**.

PASO 2 · Esperar **11 segundos** completos.

PASO 3 · **H09**: consultar diagnóstico; puede seguir **open** hasta recibir una nueva operación protegida.

PASO 4 · **H10** → **H11**: compra 1/3 **HTTP 201**; diagnóstico **half-open**, 1 éxito.

PASO 5 · **H12** → **H13**: compra 2/3 **HTTP 201**; diagnóstico **half-open**, 2 éxitos.

PASO 6 · **H14** → **H15**: compra 3/3 **HTTP 201**; diagnóstico **closed**.

PASO 7 · Revisar resultado en **DataGrip**: refrescar A+B+C+D y mirar transiciones en **Terminal 4**.

> **VERIFICACIÓN** 3 órdenes **COMPLETED**, pagos **CHARGED**, reservas **RESERVED** y envíos **SCHEDULED** nuevos. **Stock 85 → 82**. Logs **open** → **half-open** → **closed**.
>
> **VERIFICACIÓN · Terminal 4 (Order):** circuit_breaker=**shipping** old_state=**open** new_state=**half-open**; ... **half-open** new_state=**closed**

> **Interpretación** «**Los tres envíos se programaron correctamente. Shipping pasó de Open a Half-Open y, con el tercer éxito, volvió a Closed: quedó confirmada su recuperación.**»

2.7 Cierre de demostración

> **VERIFICACIÓN** Inventario esperado si no hubo envíos extra: 100 inicial → 98 tras Saga → 90 tras CB **Payment** → 82 tras CB **Shipping**.

> **Conclusión** «**La demostración confirma dos funciones diferentes: Saga revierte los pasos completados cuando una compra falla; Circuit Breaker evita llamadas repetidas a un servicio con errores y comprueba cuándo se recupera. Juntos permiten manejar fallos sin dejar operaciones inconclusas.**»

No ejecutar Run Collection, los GET opcionales **S1.02–S4.04** ni compras adicionales durante este recorrido.